

CMPS 312 – Mobile Application Development

Final Practical Exam 5 December 2024 (duration 2h)

Student Name (Student ID)

Instructions : The exam is be open book **but not open internet**. You may only access the official documentation pages and the lab materials provided. The use of **Google or AI tools is strictly prohibited**.

Task Overview:

1. Sync the cmps312-lab repository and copy the Final Exam folder to your GitHub repository. The folder contains the base project named **StreamVault** with all necessary dependencies and the testing sheet.
2. Implement a **StreamVault** app that allows users to explore movies by platforms. Manage app data in an SQLite database using the **Floor Library**.

Implementation Tasks:

1. **Entity-Relations [15]:** Create two Entity classes, **Movie** and **Platform**, each with proper annotations as outlined in Figure 1. Ensure that the IDs are auto-assigned by the database. Add foreign key annotations with cascade delete and update on the appropriate entity.

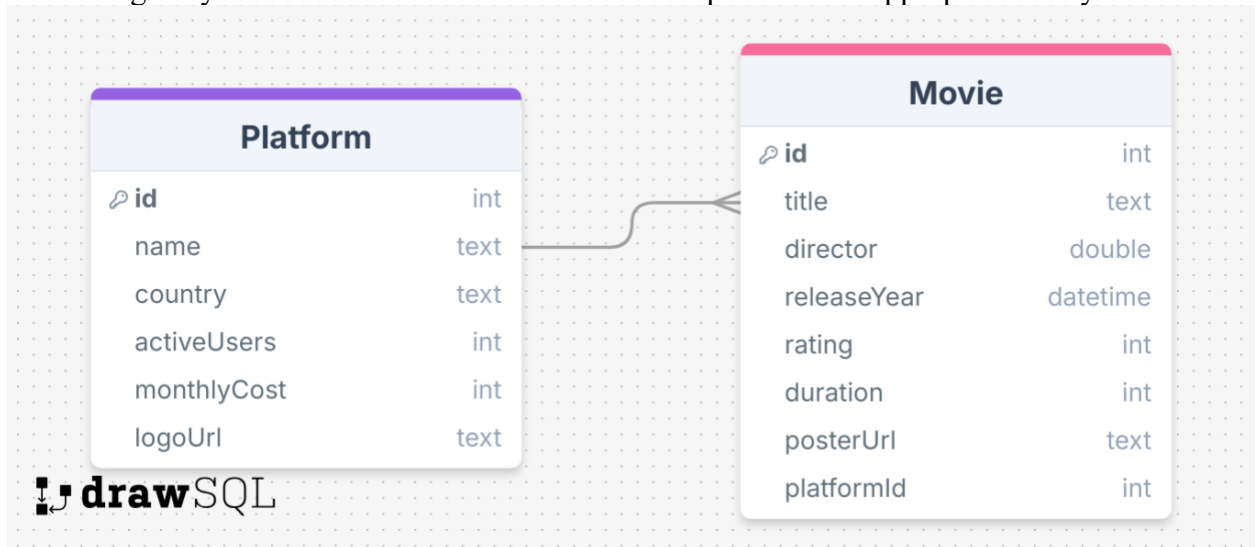


Figure 1. StreamValut Entity-Relations

2. **DatabaseView [20]:** Create a DatabaseView that provides a summary of platforms, including:
 - a. The **total number of movies** for each platform.
 - b. The **average rating** of movies for each platform.
 - c. The **average duration** of movies for each platform.

```
class PlatformSummaryView {
    final int platformId;
    final String platformName;
    final int totalMovies;
    final double averageRating;
    final double averageDuration;
}
```

```
PlatformSummaryView(this.platformId, this.platformName,
this.totalMovies, this.averageRating, this.averageDuration);
}
```



3. Create the Dao [35]

A. PlatformDao [20]: Add methods for managing platform data:

- **Insert or update a new platform.** If the platform exists update otherwise add.
- Retrieve all platforms as a stream.
- Delete a platform and all associated movies using cascading.
- Update a platform's details.
- Get **PlatformSummary** as a stream.

B. MovieDao [15]: Add methods for managing movie data:

- Add a new movie for a platform.
- Retrieve all movies associated with a **specific platform**
- Update a specific movie's details.
- Delete a movie by its id.

4. FilmFusionRepository [10]: Implement the FilmFusionRepository with all necessary methods listed above.

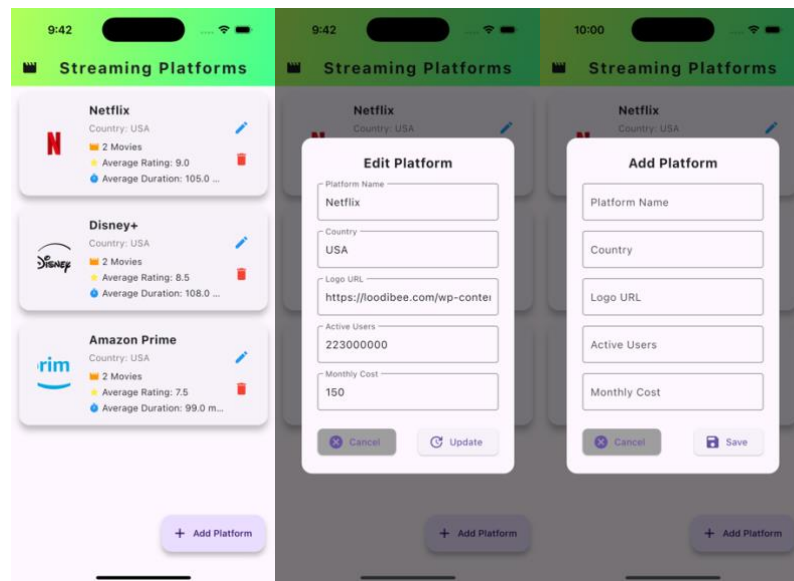
5. Initialize the database from Web [15]: Implement a utility function to fetch initial data (e.g., a JSON file with platforms and movies) from the provided API endpoints:

- Platforms API Endpoint: <https://api-8t7c.onrender.com/platforms>
- Movies API Endpoint: <https://api-8t7c.onrender.com/movies>

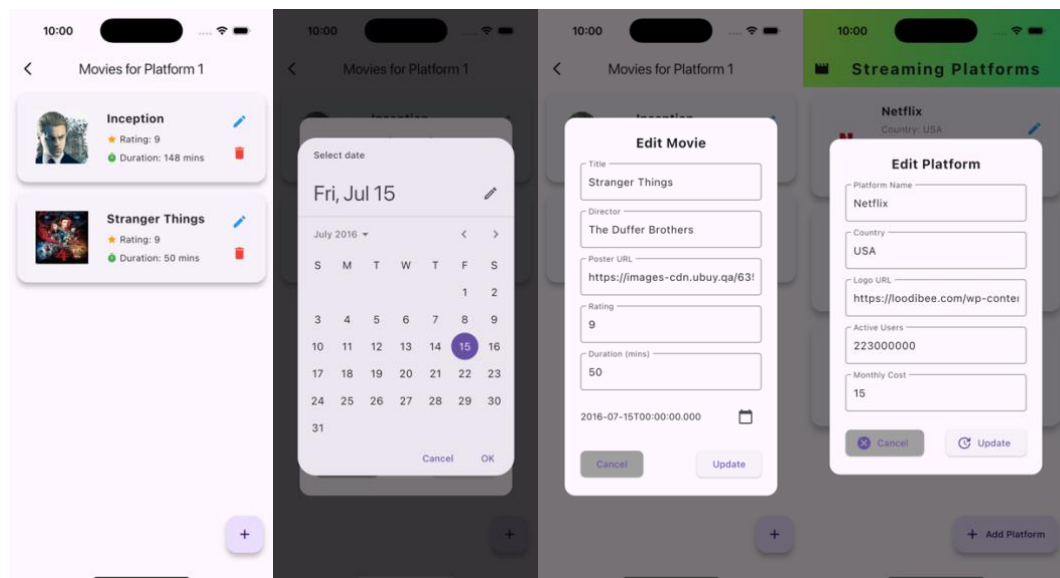
6. Implement the FilmFusionRepositoryProvider [5]

7. Test your application: If the application fails to work, 30% of the grade will be deducted, even if the rest of the code is written. Below are the application demo screens.

- Initially, you should display the populated data from the web API related to platforms and movies. In the Platform Card, display the following information: the platform name as the title of the card, the country of origin, and a dynamic summary that shows the total number of movies available on the platform along with the average movie rating using real-time data fetched through a StreamBuilder. Include action buttons for editing and deleting the platform, allowing users to interact with the card.



- Users should be able to add, update, and delete a movie under a specific platform, as well as edit and delete an entire platform. The user interface should be intuitive, enabling seamless management of platforms and their associated movies.



Grading Rubric - - In the *Functionality* column please specify either: *Working (completed x%)*, *Not Working (completed x%)* or *Not done*.

Grading Rubric:

Criteria	Possible Points
Entity-Relations Implementation	15
DatabaseView Creation	20
PlatformDao Implementation	20
MovieDao Implementation	15
FilmFusionRepository Implementation	10
Initialize Database from Web	15
Implement FilmFusionRepositoryProvider	5
Application Testing	Deduction: -30%

Possible grading for functionality:

- ***Complete and Working*** (get 70% of the assigned grade),
- ***Complete and Not working*** (lose 30% of assigned grade)
- ***Not done*** get 0.
- The remaining grade is assigned to the quality of the implementation.
- Must submit screenshots in this **Testing-GradingSheet.docx** (otherwise - 10pts).